



POTTERY DATABASE

**SETTING A DATABASE
SYSTEM FOR A
POTTERY
BUSSINESS**

**LANA NEZAR SALIM
2517247**

**MANAL AL-AMUDI
2517400**



Narrative Description

The pottery-making studio industry in the Middle East remains relatively limited in scale and visibility compared to other creative sectors. While there are a number of studios and small businesses operating across the region, many of them rely on traditional management methods and do not typically implement structured digital systems such as databases to organize their operations. This often leads to inefficiencies in handling customer information, inventory, workshop scheduling, and sales tracking.

To address this gap, we are developing a dedicated database system for a pottery business called Fokhar & Co. Fokhar & Co is a business that depends on selling ready-to-buy pottery pieces as well as a studio that offers pottery workshops and sessions where customers shape their own pieces. The purpose of this system is to streamline daily operations by centralizing data related to customers, products, bookings, and transactions. By introducing a structured database, Fokhar & Co. can improve accuracy, enhance customer experience, and support future growth, setting a model for modernization within this niche industry.

Distribution of Duties

<u>Members</u>	<u>Duties</u>
Lana Nezar	• Narrative Description • ER Model • EER Model • Relational Schema
Manal Mohammed	• Normalization • SQL • Procedure
Shared Work	• Overview of the Concept • Functional Dependencies

An Overview of the Concept

“Fokhar& Co.” : A Pottery Business that :

Settle Pottery-Making Workshops & Sessions

Sell Ready-to-Buy Pottery Pieces

4 Main Sectors :

PEOPLE

EXPERIENCE

PRODUCTION

RETAIL

Entities :

- Customer
- Instructor/Staff

- Workshops (Theoretical)
- Session (Practical)
- Booking

- Piece (Created)
- Piece Status
- Failure Log

- Product
- Sale / Invoice

ER Model

(Entity-Relationship Model)

PEOPLE

CUSTOMER

PK CustomerID

Name
PhoneNumber
Email
MembershipLevel

INSTRUCTOR / STAFF

PK InstructorID

Name
Specialization
PhoneNumber
Email

EXPERIENCE

WORKSHOP

PK WorkshopID

Title
Description
Price
Duration

SESSION

PK SessionID

FK WorkshopID
Date
Time
RoomNumber
MaxCapacity

BOOKING

PK BookingID

FK SessionID
FK CustomerID

BookingDate
PaymentStatus
NumberOfAttendees

PRODUCTION

PIECE

PK PieceID

FK CustomerID
FK InstructorID
ShapeType
WeightOfClay
EstimatedReadyDate
FK StatusID

PIECE STATUS

PK StatusID

StatusName
Description

FAILURE LOG

PK FailureID

FK PieceID
Reason
DateOfFailure

RETAIL

PRODUCT

PK ProductID

Name
Category
Price
StockQuantity

SALE / INVOICE

PK SaleID

FK CustomerID
TransactionDate
TotalAmount
PaymentMethod

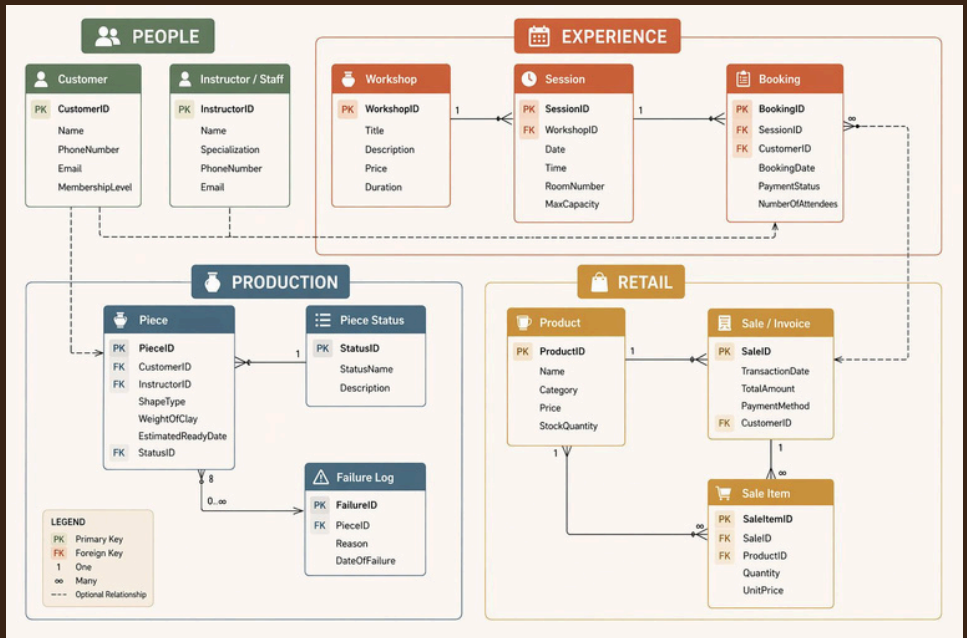
SALE ITEM

PK SaleItemID

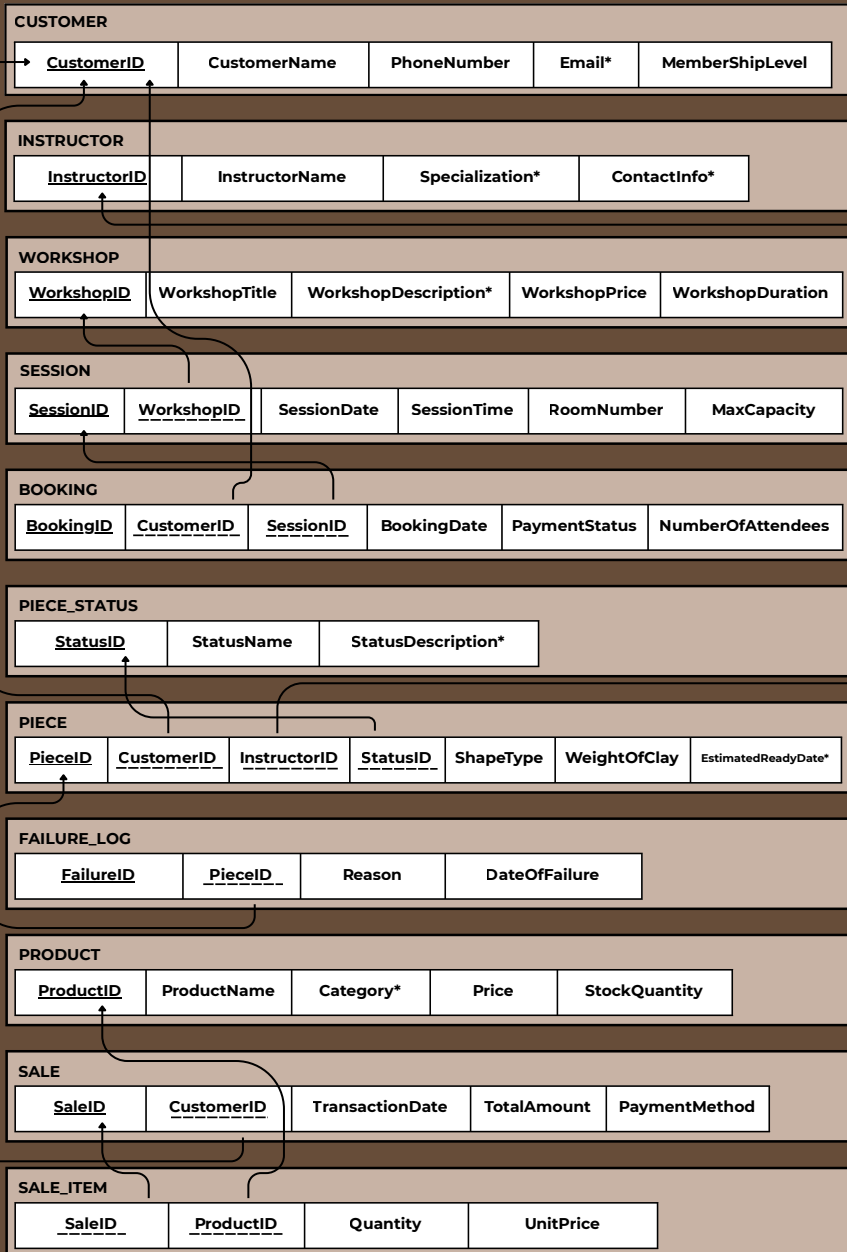
FK SaleID
FK ProductID
Quantity
UnitPrice

EER Model

(Enhanced Entity-Relationship Model)



Relational Schema



Functional Dependencies

Functional Dependencies

CUSTOMER:

CustomerID → {Name, PhoneNumber, Email, MembershipLevel}

INSTRUCTOR/STAFF:

InstructorID → {Name, Specialization, PhoneNumber, Email}

WORKSHOP:

WorkshopID → {Title, Description, Price, Duration}

SESSION:

SessionID → {WorkshopID, Description, Price, Duration}

BOOKING:

BookingID → {CustomerID, SessionID, BookingDate, PaymentStatus, NumberOfAttendees}

PIECE_STATUS:

StatusID → {StatusName, StatusDescription}

PIECE:

PieceID → {CustomerID, InstructorID, StatusID, ShapeType, WeightOfClay, EstimatedReadyDate}

FAILURE_LOG:

FailureID → {PieceID, Reason, DateOfFailure}

PRODUCT:

ProductID → {ProductName, Category, Price, StockQuantity}

SALE:

SaleID → {CustomerID, TransactionDate, TotalAmount, PaymentMethod}

SALE_ITEM(Composite Key):

(SaleID, ProductID) → {Quantity, UnitPrice}

Normalization

1NF:

1. All relations have a unique primary key.
 2. No repeating groups or multi-valued attributes exist.
 3. No composite attributes exist; all attributes are atomic.
- The schema already satisfies the 1NF rules, as all attributes are atomic.

2NF:

1. Remove partial functional dependencies.
- All relations have either a single primary key or a composite primary key with full dependency, ensuring all non-key attributes are fully dependent on the primary key. The schema already satisfies the 2NF rules.

3NF:

1. Remove transitive dependencies.
- There are no transitive dependencies in the schema; all non-key attributes are dependent only on the primary key. For example, in the "Sale Item" relation, UnitPrice is stored as a snapshot of the price at the time of the transaction to maintain historical integrity and prevent transitive dependency on the "Product" relation. The schema already satisfies the 3NF rules.

SQL

(Structured Query Language)

CREATE TABLES

```
1 CREATE TABLE Product (
2     ProductID INT PRIMARY KEY,
3     Name VARCHAR2(100),
4     Category VARCHAR2(50),
5     Price NUMBER(10, 2),
6     StockQuantity INT
7 );

1 CREATE TABLE PieceStatus (
2     StatusID INT PRIMARY KEY,
3     StatusName VARCHAR2(50),
4     Description VARCHAR2(200)
5 );

1 CREATE TABLE Customer (
2     CustomerID INT PRIMARY KEY,
3     Name VARCHAR2(100),
4     PhoneNumber VARCHAR2(20),
5     Email VARCHAR2(100),
6     MembershipLevel VARCHAR2(50)
7 );

1 CREATE TABLE Booking (
2     BookingID INT PRIMARY KEY,
3     SessionID INT REFERENCES Session_T(SessionID),
4     CustomerID INT REFERENCES Customer(CustomerID),
5     BookingDate DATE,
6     PaymentStatus VARCHAR2(50),
7     NumberOfAttendees INT
8 );
```

```
1 CREATE TABLE Instructor (
2     InstructorID INT PRIMARY KEY,
3     Name VARCHAR2(100),
4     Specialization VARCHAR2(100),
5     PhoneNumber VARCHAR2(20),
6     Email VARCHAR2(100)
7 );

1 CREATE TABLE Session_T (
2     SessionID INT PRIMARY KEY,
3     WorkshopID INT REFERENCES Workshop(WorkshopID),
4     SessionDate DATE,
5     SessionTime VARCHAR2(20),
6     RoomNumber VARCHAR2(10),
7     MaxCapacity INT
8 );

1 CREATE TABLE Workshop (
2     WorkshopID INT PRIMARY KEY,
3     Title VARCHAR2(100),
4     Description VARCHAR2(200),
5     Price NUMBER(10, 2),
6     Duration VARCHAR2(50)
7 );
```

```
1 CREATE TABLE FailureLog (
2     FailureID INT PRIMARY KEY,
3     PieceID INT REFERENCES Piece(PieceID),
4     Reason VARCHAR2(200),
5     DateOfFailure DATE
6 );

1 CREATE TABLE Sale (
2     SaleID INT PRIMARY KEY,
3     CustomerID INT REFERENCES Customer(CustomerID),
4     TransactionDate DATE,
5     TotalAmount NUMBER(10, 2),
6     PaymentMethod VARCHAR2(50)
7 );

1 CREATE TABLE SaleItem (
2     SaleItemID INT PRIMARY KEY,
3     SaleID INT REFERENCES Sale(SaleID),
4     ProductID INT REFERENCES Product(ProductID),
5     Quantity INT,
6     UnitPrice NUMBER(10, 2)
7 );

1 CREATE TABLE Piece (
2     PieceID INT PRIMARY KEY,
3     CustomerID INT REFERENCES Customer(CustomerID),
4     InstructorID INT REFERENCES Instructor(InstructorID),
5     ShapeType VARCHAR2(100),
6     WeightOfClay NUMBER(10, 2),
7     EstimatedReadyDate DATE,
8     StatusID INT REFERENCES PieceStatus(StatusID)
9 );
```

SQL

(Structured Query Language)

INSERT TABLES

```
1  -- Inserting into Customer
2  INSERT INTO Customer VALUES (1, 'Ahmed Ali', '0501112222', 'ahmed@email.com', 'Gold');
3  INSERT INTO Customer VALUES (2, 'Sara Hassan', '0503334444', 'sara@email.com', 'Silver');
4  INSERT INTO Customer VALUES (3, 'Khalid Omar', '0505556666', 'khalid@email.com', 'Bronze');
5  INSERT INTO Customer VALUES (4, 'Noura Fahad', '0507778888', 'noura@email.com', 'Gold');
6  INSERT INTO Customer VALUES (5, 'Mona Saad', '0509990000', 'mona@email.com', 'Silver');
7
8
9  -- Inserting into Workshop
10 INSERT INTO Workshop VALUES (101, 'Pottery Basics', 'Learn the basics of clay', 150.00, '3 Hours');
11 INSERT INTO Workshop VALUES (102, 'Wheel Throwing', 'Advanced wheel techniques', 250.00, '4 Hours');
12 INSERT INTO Workshop VALUES (103, 'Glazing Art', 'Coloring and glazing', 100.00, '2 Hours');
13 INSERT INTO Workshop VALUES (104, 'Sculpting', 'Hand sculpting figures', 300.00, '5 Hours');
14 INSERT INTO Workshop VALUES (105, 'Kids Pottery', 'Fun with clay for kids', 80.00, '1.5 Hours');
15
16 -- Inserting into Product
17 INSERT INTO Product VALUES (201, 'Clay Vase', 'Vases', 45.00, 10);
18 INSERT INTO Product VALUES (202, 'Ceramic Mug', 'Tableware', 25.00, 25);
19 INSERT INTO Product VALUES (203, 'Decorative Plate', 'Decor', 60.00, 5);
20 INSERT INTO Product VALUES (204, 'Pottery Tool Set', 'Tools', 120.00, 15);
21 INSERT INTO Product VALUES (205, 'Mini Cactus Pot', 'Decor', 15.00, 40);
22
23 -- Inserting into Session_T
24 INSERT INTO Session_T VALUES (1, 101, TO_DATE('2026-05-10', 'YYYY-MM-DD'), '10:00 AM', 'Room');
25 INSERT INTO Session_T VALUES (2, 102, TO_DATE('2026-05-11', 'YYYY-MM-DD'), '02:00 PM', 'Room');
26 INSERT INTO Session_T VALUES (3, 103, TO_DATE('2026-05-12', 'YYYY-MM-DD'), '09:00 AM', 'Room');
27 INSERT INTO Session_T VALUES (4, 104, TO_DATE('2026-05-13', 'YYYY-MM-DD'), '04:00 PM', 'Room');
28 INSERT INTO Session_T VALUES (5, 101, TO_DATE('2026-05-14', 'YYYY-MM-DD'), '11:00 AM', 'Room');
29
30 -- Inserting into Booking
31 INSERT INTO Booking VALUES (1, 1, 1, TO_DATE('2026-05-01', 'YYYY-MM-DD'), 'Paid', 1);
32 INSERT INTO Booking VALUES (2, 2, 2, TO_DATE('2026-05-02', 'YYYY-MM-DD'), 'Pending', 2);
33 INSERT INTO Booking VALUES (3, 3, 3, TO_DATE('2026-05-03', 'YYYY-MM-DD'), 'Paid', 1);
34 INSERT INTO Booking VALUES (4, 1, 4, TO_DATE('2026-05-04', 'YYYY-MM-DD'), 'Paid', 1);
35 INSERT INTO Booking VALUES (5, 5, 5, TO_DATE('2026-05-05', 'YYYY-MM-DD'), 'Paid', 3);
36
```

SQL

(Structured Query Language)

1st QUERY (Join + Where + Order By)

```
1 SELECT c.Name, w.Title, b.BookingDate, b.PaymentStatus
2 FROM Customer c
3 JOIN Booking b ON c.CustomerID = b.CustomerID
4 JOIN Session_T s ON b.SessionID = s.SessionID
5 JOIN Workshop w ON s.WorkshopID = w.WorkshopID
6 WHERE b.PaymentStatus = 'Paid'
7 ORDER BY b.BookingDate DESC;
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.127 seconds

	NAME	TITLE	BOOKINGDATE	PAYMENTSTATUS
1	Mona Saad	Pottery Basics	5/5/2026, 12:00:00	Paid
2	Noura Fahad	Pottery Basics	5/4/2026, 12:00:00	Paid
3	Khalid Omar	Glazing Art	5/3/2026, 12:00:00	Paid
4	Ahmed Ali	Pottery Basics	5/1/2026, 12:00:00	Paid

2nd QUERY (Group By + Aggregate Functions + Join)

```
1 SELECT W.Title, SUM(B.NumberOfAttendees) AS Total_Attendees
2 FROM Workshop W
3 JOIN Session_T S ON W.WorkshopID = S.WorkshopID
4 JOIN Booking B ON S.SessionID = B.SessionID
5 GROUP BY W.Title;
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.127 seconds

	TITLE	TOTAL_ATTENDEE
1	Pottery Basics	5
2	Wheel Throwing	2
3	Glazing Art	1

SQL

(Structured Query Language)

3rd QUERY (Subquery + Where + Join + Aggregate Functions)

```
1 SELECT P.Name, P.Category, P.Price
2 FROM Product P
3 WHERE P.Price > (SELECT AVG(Price) FROM Product)
4 ORDER BY P.Price ASC;
```

Query result Script output DBMS output Explain Plan SQL history

  Download ▾ Execution time: 0.011 seconds

	NAME	CATEGORY	PRICE
1	Decorative Plate	Decor	60
2	Pottery Tool Set	Tools	120

4th QUERY (Join + Aggregate Functions + Group By + Order By)

```
1 SELECT W.Title, S.RoomNumber, S.MaxCapacity
2 FROM Workshop W
3 JOIN Session_T S ON W.WorkshopID = S.WorkshopID
4 WHERE S.RoomNumber = 'Room A'
5 ORDER BY S.MaxCapacity DESC;
6
7
```

Query result Script output DBMS output Explain Plan SQL history

  Download ▾ Execution time: 0.012 seconds

	TITLE	ROOMNUMBER	MAXCAPACITY
1	Glazing Art	Room A	12
2	Pottery Basics	Room A	10

Procedure

1st Procedure

```
[SQL Worksheet]*  ▶  ⚙  📄  📋  Aa  🗑

1  CREATE OR REPLACE PROCEDURE GetCustomerName (
2  |   p_CustID IN NUMBER
3  ) AS
4  |   v_Name VARCHAR2(100);
5  BEGIN
6  |   SELECT Name INTO v_Name
7  |   FROM Customer
8  |   WHERE CustomerID = p_CustID;
9
10 |   DBMS_OUTPUT.PUT_LINE('Customer Name is: ' || v_Name);
11 EXCEPTION
12 |   WHEN NO_DATA_FOUND THEN
13 |       DBMS_OUTPUT.PUT_LINE('No customer found with ID: ' || p_CustID);
14 END;
```

```
1  BEGIN
2  |   GetCustomerName(1);
3  END;
```

Query result **Script output** DBMS output Explain Plan SQL history



```
SQL> BEGIN
      GetCustomerName(1);
      END;
```

Customer Name is: Ahmed Ali

Procedure

2nd Procedure

```
[ SQL Worksheet ]*  ▶  ⌵  🔍  📄  ⌵  Aa  🗑️

1  CREATE OR REPLACE PROCEDURE UpdateProductPrice (
2      p_ProductID IN NUMBER,
3      p_NewPrice IN NUMBER
4  )
5  AS
6  BEGIN
7      UPDATE Product
8      SET Price = p_NewPrice
9      WHERE ProductID = p_ProductID;
10
11  IF SQL%ROWCOUNT = 0 THEN
12      DBMS_OUTPUT.PUT_LINE('No product found with this ID.');
```

```
1  BEGIN
2      UpdateProductPrice(1, 150.00);
3  END;
```

Query result

Script output

DBMS output

Expla



```
SQL> BEGIN
      UpdateProductPrice(1, 150.00);
END;
```

Price updated successfully.

END OF PROJECT

POTTERY DATABASE

SUPERVISED BY: DR. HALA BAKHSH

**DATABASE COURSE INSTRUCTOR: DR. MASHAEL
KHAYYAT**

